

XS rules functionality

1. [How to define a rule](#)
2. [Enabling and disabling of rules](#)
3. [When are rules executed](#)
4. [High frequency](#)
5. [Rule ordering](#)
6. [Rule groups](#)
7. [Run immediately](#)
8. [Misc](#)

i In short: XS rules is the mechanism which determines what script code should be executed when.

Each game update we run the AI scripts attached to the players and potentially a trigger script attached to the map/scenario. But we of course don't run the entirety of each script every game update. We only run the code we want to run. And rules are our way of determining what code we actually want to run each update.

i AI Example: looking for if we should build a Wonder while we're in the Archaic age makes no sense. So the rule that builds Wonders is not enabled while we're in Archaic. While the rule that tries to build a Temple so that we can age up to the Classical Age is enabled.

At any point in the game the rules that you have on “*active*” determines what script code is potentially executed each game update. You have to make sure that the right set of rules is active at the right time.

This document will outline both how to define a default rule as well as how to modify the behavior of a rule.

1. How to define a rule

```
1 rule myRuleName
2 {
3 }
```

This is a plain rule, meaning it has no behavior specifically defined everything is default.

```
1 rule myRuleName
2 // Here go all the modifiers that are explained on this page.
3 // Their order does not matter.
4 {
5 }
```

2. Enabling and disabling of rules

A rule is by default inactive (disabled), however as can be read in the coding standards we should always specifically define if it's active or inactive for readability.

```
1 rule iAmActive
2 active
3 {
4 }
5
```

```

6 rule iAmInactive
7 inactive
8 {
9 }

```

After the initial state of active/inactive there are several XS syscalls that can be used to enable/disable a rule/rule group.

3. When are rules executed

The following section only applies to rules that are currently marked as “*active*”.

Each game update we have an internal update time in **milliseconds**. And each rule has three timers saved:

- Last execution time in **milliseconds**.
- Min execution interval in **milliseconds**.
 - Defaults to 1 millisecond, so basically highFrequency (read about it below)
- Max execution interval in **milliseconds**. (only relevant for AI)
 - Defaults to 60000 milliseconds (1 minute)

```

1 rule whenDoIExecute
2 minInterval 5 (sets the time in seconds, internally this number will be * 1000)
3 maxInterval 10 (sets the time in seconds, internally this number will be * 1000)
4 {
5     // I at the earliest execute every 5 seconds, max 10 sec between each execution.
6 }
7
8 rule whenDoIExecuteMS
9 minIntervalMS 3000 (instantly sets it in ms)
10 maxIntervalMS 7000 (instantly sets it in ms)
11 {
12     // I at the earliest execute every 3 seconds, max 7 sec between each execution.
13 }

```

In principal this applies:

```

1 if (last execution time + min execution interval < internal update time)
2 {
3     executeRule(rule);
4 }

```

For triggers this is always how it goes, any rule that has its min interval reached will be executed that game update. But for the AI it works differently, because of performance reasons we can’t just blindly execute all rules that have met their min interval. Per AI we only want to spend ~5ms evaluating XS code, or with too many AI the game starts lagging. Thus if we’ve already spent >5ms in the XS code for an AI we skip any subsequent rules this game update, even if they would qualify via the min execution interval system.

However, we have a method to force a rule to execute even if it would mean going above the 5ms preferred limit. This is where the max interval mechanic comes in.

```

1 if (last execution time + max execution interval < internal update time)
2 {
3     executeRule(rule);
4 }

```

Example: let’s say we have a rule with a min interval of 1 and a max interval of 2. Due to some circumstance the overall rule execution is very slow, and our example rule always ends up being skipped because the rules that come before it

took way too long. Now if the game update time has advanced so much that it's more or equal to 2 seconds ago that our example rule was executed, it's not skipped again but it's forcefully executed because we're respecting the provided max interval.

We can also force rules to be executed during the next game update via XS syscalls: *xsRuleIgnoreIntervalOnce* or *xsRuleGroupIgnoreIntervalOnce* can be used to achieve this.

4. High frequency

High frequency is a way to indicate that a rule should be executed each frame. How that works under the hood is that the *minInterval* and *maxInterval* of the rule are set to 1ms, which forces its execution each game update. **Note:** this doesn't force the rule to be executed on the first game update, read below about *runImmediately*.

```
1 rule iWillAlwaysBeExecuted
2 highFrequency
3 {
4 }
```

5. Rule ordering

After reading the “*When are rules executed*” section you may be wondering in what fashion we loop through all the XS rules. Because if a rule is all the way at the end of the loop it has a higher chance of being skipped if we're an AI. Or if we're going to execute 2 rules during the same game update, which one goes first?

This is where the priority system comes in. A rule can have a priority between 100-0 (inclusive). All rules that are declared in your script are parsed in the order in which they are defined. Afterwards a prioritized list is made. If a rule has not been given a priority it gets the default 50 assigned. If a rule has the same priority as another rule, then the rule that got defined first in the script ends up higher in the prioritized list.

```
1 rule example1
2 {
3 }
4 rule example2
5 {
6 }
7 rule example3
8 {
9 }
10 rule example4
11 priority 51
12 {
13 }
14 rule example5
15 priority 51
16 {
17 }
18
19 // Update list will look like this:
20 // Higher prio goes to the top, but equal prio the order of definition is maintained.
21 example4
22 example5
23 example1
24 example2
25 example3
```

6. Rule groups

Rule groups are a tool to help with rule enabling/disabling. Instead of having to do it all one by one and potentially patch up multiple places when you add a new rule you can instead use rule groups.

```
1 rule iAmInAGroup
2 group group1
3 {
4 }
5
6 rule iAmAlsoInAGroup
7 group group1
8 {
9 }
```

We now have two rules that are both in the same rule group. I can now use `xsEnableRuleGroup("group1");` to enable them both. Or use other syscalls related to rule groups to modify everything quickly.

 AI uses this functionality very often because there are some pretty logical situations for groups. Like each age up new mechanics become available and thus we have a rule group for each age. Enable military handling related rules in Classical, enable Fortress related rules in Heroic etc...

7. Run immediately

If you want a rule to execute on the first frame of the game, the same frame as the main function is executed, you must use this mechanic.

```
1 rule iWillBeExecutedVerySoon
2 minInterval 15
3 runImmediately
4 {
5 }
```

Even though the `minInterval` is set to 15 it will still be executed on the first frame because of the `runImmediately` definition.

8. Misc

- Names of rules / functions must be in ASCII characters.
- Setting `highFrequency` combined with a `minInterval`/`maxInterval` makes no sense since both will adjust the same data, but it won't compiler error.
- Since `maxInterval` defaults to 60 it means that if you have a rule defined without a `maxInterval` but do have a `minInterval` defined which is ≥ 60 , the rule will **always** be executed the moment it passes the `minInterval` since the `maxInterval` will force it.
- RMS runtime has no rules because it's all executed at once, completely removing the need for rules.